

FOL: Bridging Object-Oriented and Functional Programming via the Metaobject Protocol

Frank Adrian
Ancar Technology
Olathe, KS, USA

frank.adrian314159@gmail.com

Abstract

Hickey [18] omitted CLOS from Clojure as a complexity hazard incompatible with immutability. FOL (Functional Object Lisp) uses the Metaobject Protocol (MOP) to back instances with persistent structures, preserving the full CLOS model while enforcing value semantics. FOL contributes a persistent metaclass with hybrid storage and lazy migration, and a specificity-ordered predicate dispatch model.

Slot reads incur 1.2–2.0× overhead; slot updates cost 30–250× in isolation. At typical workload scales, FOL is 5–43× slower than mutable CLOS; however, the fixed $O(\log n)$ structural-copy cost avoids heap fragmentation, and at extreme scale the persistent heap outperforms a fragmented mutable baseline, yielding a 3.8× runtime advantage in logic simulation. Predicate dispatch uses a level-based ordering sorted once at load time in $O(ND \log N)$ (D = hierarchy depth), avoiding the $O(N^2)$ theorem-prover overhead of subsumption systems; the trade-off is an $O(N)$ per-call linear scan (vs. $O(1)$ cached dispatch), with closed defn dispatch matching hand-written cond cascade performance.

CCS Concepts

• **Software and its engineering** → **Constraints; Classes and objects; Functional languages; Object oriented languages; Extensible languages.**

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP

1 Introduction

Common Lisp’s object system (CLOS) is extensible, but its mutable objects create three problems.

P1 — Speculative update cost. Speculative workloads must copy objects or maintain undo logs. Persistent alternatives (FSet [5], Sycamore [9]) lack CLOS integration and generic-function dispatch. Isolated slot updates cost 30–248×, but $O(\log n)$ structural sharing bounds per-update cost regardless of the number of live objects in the system; at sufficient scale, path-copying avoids the heap fragmentation that degrades mutable baselines (Section 5.5).

P2 — Schema evolution labor. Renaming slots requires manual update-instance-for-redefined-class methods; the standard hook lacks automatic forwarding. FOL’s `:alias` annotation recovers renamed slots at first access, composing multi-hop chains in a single pass (Section 5.2.2).

P3 — Inexpressive predicate dispatch. CLOS dispatch is type-limited; value-level conditions require manual cond guards, losing extensibility. Libraries such as `filtered-functions` [26] extend

this with logical subsumption but require $O(N^2)$ theorem-prover calls. FOL’s level-based ordering (predicates > types > destructuring > wildcards) is decidable at load time in $O(ND \log N)$ (D = hierarchy depth); per-invocation overhead matches a manual cond cascade ($\approx 1.6\times$ vs. CLOS; Section 5.2.1).

FOL (Functional Object Lisp) addresses these via the MOP [20]: every class instance is backed by persistent data structures, and specificity-ordered predicate dispatch integrates with standard method combinations. Slot mutation is replaced by `assoc/dissoc`; other CLOS interfaces behave normally (Table 1). The contributions compose: Listing 4 (Section 4) shows a predicate gating on runtime values while reading persistent state. Listing 3 shows $O(1)$ update discard; the `:` around qualifier and the immutability invariant enable this pattern without copying.¹

The contributions of this paper are:

- (1) A **persistent metaclass system** (Section 2) with a hybrid native/trie storage strategy and lazy alias-based schema migration via multi-hop chain replay (Section 4.2).
- (2) A **specificity-ordered predicate dispatch model** (Section 4): a level-based ordering relation that is decidable at load time without theorem-prover calls, with integration into standard CLOS method combinations.

2 Architecture

2.1 Object Implementation

Slot storage uses a hybrid strategy: objects with ≤ 8 slots use native CLOS slots, while wider objects overflow into a 32-way persistent trie. Based on Lanza’s Java class distributions [21], $\approx 80\%$ of classes have ≤ 8 slots. Table 7 shows $T = 8$ also minimizes modeled update cost for 8-slot objects. All redefinitions are handled lazily at the next object mutation (`assoc`, `dissoc`, etc.); reads of renamed slots resolve via the alias map without triggering migration.

The persistent-class metaclass (Table 1) ensures no slot value is modifiable post-initialization; standard CLOS mutation paths raise runtime errors. `assoc/dissoc` return a structurally equal ($=$) fresh instance sharing structure with the original, preserving `%metadata`; references are pointers to immutable snapshots, decoupling state from identity (*eq*).

A reserved `%metadata` slot stores non-structural annotations; objects differing only in metadata are $=$ and hash identically. Change-count slots used for diff tracking (Section 4) are structural and visible to $=$; callers wishing to track changes invisibly should store them in `%metadata` instead.

¹FOL addresses both dimensions of the Expression Problem [31]: new generic functions can be added over existing classes; new clauses can be added for new types. Static safety for the new-types dimension requires type-system guarantees FOL does not provide.

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
slot-value-using-class	Redirected	Reads from base object or overflow trie
(setf slot-value-using-class)	Guarded	Blocks post-init writes
slot-boundp-using-class	Redirected	Checks object slot membership
slot-makunbound-using-class	Blocked	Immutability invariant — use dissoc to unbind slots
validate-superclass	Extended	Cross-metaclass mixing
initialize-instance	Extended	Populates base object and overflow trie from initargs; stamps %schema-version (an integer matching the class’s current version counter) at birth, enabling update-instance-for-redefined-class to detect stale instances
reinitialize-instance	Extended (:before on metaclass)	Snapshots current alias-map, overflow indices, and version counter into a class-version struct before each redefinition; advances the version counter
compute-slots	Specialized	Implements hybrid native/trie layout
update-instance-for-redefined-class	Implemented (:after)	Multi-hop chain replay: composes alias-maps from version chain back to birth version, recovering renamed values (Sec. 4.2)
change-class	Omitted	Changing a live object’s class violates immutability; callers construct a new instance of the target class instead (requires compatible metaclasses)

Method combination uses standard CLOS execution semantics (:before/:after/:around), with method ordering determined by $\prec_{disp}$ in place of the class precedence list.

2.2 Collections and Runtime

FOL implements persistent **Vectors** (32-way tries) [24], **Maps/Sets** (32-way HAMTs) [3], and **Sorted collections** (32-way B-trees). Branching factor 32 minimizes depth and optimizes updates. For keyword keys, HAMT lookup is $\approx 1.3\times$ faster than gethash,² as EQL comparison avoids the general hashing path.

FOL transpiles to Common Lisp (compiled by SBCL) via four phases: **reader expansion**, **macro expansion**, **code generation**, and **compilation**.

2.3 Space Complexity

FOL collections and identity types (Atoms, Refs, Agents) hold only the current value; old snapshots are GC-eligible as soon as no live reference remains.³ Each assoc copies one root-to-leaf path— $O(\log_{32} N)$ new nodes, not $O(N)$. Short-lived intermediates are reclaimed in SBCL’s nursery, keeping FOL GC overhead at 1.5% at the scale of our largest ablated benchmark (vs. 0.4% for the mutable CL baseline).

3 Features

FOL adopts Clojure’s literal syntax and sequence APIs and provides: **Transducers** [19] decouple algorithm logic from collection type. **Lazy sequences** defer computation, enabling infinite sequences. **Atoms** provide compare and swap-based mutable references; **Refs**

provide STM with optimistic concurrency; **Agents** handle asynchronous updates. **Transient builders** reduce bulk update overhead via thread-local mutation; W writes cost $O(W \log_{32} N)$ time but maintain only $O(\log_{32} N)$ peak live dirty nodes, committed at finalization. **Macros** follow Clojure-style expansion.

Generic functions dispatch by arity, then specificity (Section 4). **Closed dispatch** (defn) emits a defun with a static cond cascade; the clause set is fixed when the defining file is loaded and cannot be extended thereafter. **Open dispatch** (defmethod) supports CLOS method combinations and clause addition after the defining file is loaded. As defn and defmethod compete for the function slot, programmers must choose one per name; defining both signals a compile-time error. Table 2 summarises the decision criteria.

Table 2: Choosing defn vs. defmethod

Requirement	defn	defmethod
Full clause set known at definition time	✓	✓
Downstream code may add clauses	×	✓
call-next-method needed	×	✓
:before/:after/:around qualifiers	×	✓
Minimum dispatch overhead (static cond)	✓	×

4 Predicate Dispatch

Figure 1 formalizes FOL’s multi-pattern dispatch syntax.

²Mean of 10^6 single-key lookups on a 1,000-entry map, SBCL 2.6.0 x86-64. At branching factor 32, a 1,000-entry map has depth ≤ 2 ; the speedup reflects EQL-path avoidance rather than depth reduction and may not scale linearly to larger maps.

³User-managed versioning is available: callers may hold any prior assoc root; structural sharing ensures retained versions cost only $O(\log_{32} N)$ extra nodes.

```

defn ::= (defn name (s-clause | m-clause)) | (fn (s-clause | m-clause))
defg/m ::= (defgeneric name...) | (defmethod name qualifier2 (s-clause | m-clause))
s-clause ::= [param*] body+
m-clause ::= {(symbol param | :keys [(symbol | (sym param))*]*} body+
param ::= symbol | type | (symbol type) | (symbol (fn arg*)) | destr
destr ::= [param*] | { :keys [(symbol | (sym param))*]* }
type ::= <type-name>
qualifier ::= :before | :after | :around

```

Figure 1: FOL Multi-Pattern Dispatch Grammar (compressed)

4.1 Dispatch Ordering Rules

FOL’s predicate specializers (`(var (fn args...))`) evaluate as `(fn var args...)` at dispatch time; `(x (even?))` constrains value space beyond class membership, so predicates are more specific than types. Specificity categories (Table 3) order patterns by constraint strength (Level $3 > 2 > 1 > 0$), resolving ties via definition order. $\prec_{\text{disp}}$ conservatively approximates semantic specificity: Level-3 predicates fire before Level-2 types even for superset extensions. Use explicit guards—(and `(<integer>)` `(even?)`)—to clarify domains. Level 2 uses semantic subclassing (Spec-Type) for single parameters and first-parameter lexicography for multi-parameter patterns; Level 1 uses arity containment (Spec-Length/Spec-Fixed). For example consider the following code:

```

(defn classify
  ([x <integer>] :any-integer) ; intended: catch all integers
  ([x (even?)] :even))       ; intended: special case for even
                             ; numbers

```

FOL reorders this to `even?` first:

```

(cond
  ((even? x) :even)
  ((typep x (find-class '<integer>)) :any-integer))

```

Even integers now never reach `:any-integer`. FOL’s `even?` returns `nil` for non-integers, so the reordered guard is safe; user-defined predicates must maintain this totality discipline. The fix is to write clauses matching the fixed ordering:

```

(defn classify
  ([x (even?)] :even) ; catches even integers
  ([x <integer>] :other-int)) ; catches remaining integers

```

Shadowed clauses are detected at definition time (`defn`) or at each method-addition point (`defgeneric`): when a clause is statically dominated by a more-specific clause under $\prec_{\text{disp}}$, a continuable warning is signaled. This trades theorem-prover exactness for bounded compilation.

Definition-order reliance creates modularity hazards at the REPL or across libraries. We are exploring explicit ordering mechanisms, analogous to Clojure’s `prefer-method`.

All method qualifiers (`:before`/`:after`/`:around`) follow $\prec_{\text{disp}}$; standard CLOS execution semantics (most-specific-first for `:before`, least-specific-first for `:after`) are preserved.

For compound predicates, $\mathcal{L}(\text{and } p_1 \dots p_n) = \max_i \mathcal{L}(p_i)$: a conjunction of type predicates is Level 2; a mixed conjunction is Level 3. This is a conservative upper bound (the conjunction may be semantically more specific—see deficiency note below—but never less). Disjunction is excluded: (or `p` `q`) would be shadowed by separate `p` and `q` clauses. Negation is excluded because (not `p`)’s specificity

Table 3: Predicate Specificity Categories

Level	Category	Example
3	Predicates	<code>(x (even?))</code>
2	Types	<code>(x <integer>)</code>
1	Destructuring	<code>[x y]</code>
0	Wildcard	<code>x</code>

is unrelated to $\mathcal{L}(p)$; ordering it requires knowing the complement domain.

Known deficiency — type-annotated conjunctions. (and `(<integer>)` `(prime?)`) receives Level 3 ($\max(2, 3) = 3$), the same level as the bare predicate `(prime?)` alone, even though the conjunction is strictly more constrained. The syntactic calculus cannot determine this semantic subsumption without a theorem prover. The only current mitigation is definition order, which introduces a modularity hazard: if multiple libraries extend the same generic function with overlapping Level-3 predicates, the behavior depends on load order. A future composite level for type-annotated predicates (e.g., `(x <integer> (prime?))`) could recover automatic ordering without a theorem prover by treating the type as a static guard.

4.1.1 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using the inference rules summarized in Table 4. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p . **Convention:** $p_1 \prec p_2$ means p_1 fires before p_2 —i.e., p_1 is more specific. Higher levels are more specific, so Spec-Level concludes $p_1 \prec p_2$ when $\mathcal{L}(p_1) > \mathcal{L}(p_2)$.

Spec-Prefix applies when first elements are strictly ordered ($p_1 \prec q_1$); Spec-Tail recurses on the remainder when heads are level-equal; the two rules are disjoint. Spec-Type handles within-Level-2 ordering via the class hierarchy DAG; multi-parameter patterns apply the rules lexicographically position by position.

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $\text{idx}(p)$ denote the definition index of the method clause containing p .

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge \text{idx}(p_1) < \text{idx}(p_2))$$

Patterns of different arity are incomparable; dispatch selects the arity group first, then applies $\prec_{\text{disp}}$, which is total on same-arity patterns (Lemma 1). Definition-order tie-breaking aligns with Lisp’s interactive workflow but introduces the modularity hazards discussed in Section 4.1.4.

Table 4: Specificity Inference Rules

Rule	Formalization
Spec-Level	$\frac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$
Spec-Type	$\frac{C_1 \sqsubset C_2}{(x\ C_1) \prec (x\ C_2)}$
Spec-Prefix	$\frac{p_1 \prec q_1}{[p_1, \dots] \prec [q_1, \dots]}$
Spec-Tail	$\frac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \quad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$
Spec-Length	$\frac{\mathcal{L}(p_i) = \mathcal{L}(q_i) \ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n, p_{n+1}] \prec [q_1, \dots, q_n]}$
Spec-Fixed	$\frac{\mathcal{L}(p_i) = \mathcal{L}(q_i) \ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n \ \& \ r]}$
Spec-Map-Prefix	$\frac{p_1 \prec q_1}{\{k_1\ p_1, \dots\} \prec \{k_1\ q_1, \dots\}}$
Spec-Map-Tail	$\frac{p_1 \sim q_1 \quad \{k_2 \dots\} \prec \{k_2 \dots\}}{\{k_1\ p_1, k_2\ p_2, \dots\} \prec \{k_1\ q_1, k_2\ q_2, \dots\}} \quad (\sim = \text{same level and same syntactic form})$
Spec-Keys	$\frac{\text{keys}(q) \sqsubseteq \text{keys}(p)}{p \prec q}$

LEMMA 4.1 (TOTALITY). $\prec_{\text{disp}}$ is a strict total order on same-arity patterns.

PROOF. $\prec$ is irreflexive: every rule requires a strict inequality (level, element position, key set, or subclass) that is false when $p_1 = p_2$. Asymmetry of $\prec_{\text{disp}}$ follows from irreflexivity and transitivity: if $p_1 \prec_{\text{disp}} p_2$ and $p_2 \prec_{\text{disp}} p_1$ held simultaneously, transitivity would give $p_1 \prec_{\text{disp}} p_1$, contradicting irreflexivity. $\prec$ is transitive: within a level the rules encode a lexicographic comparison on position, arity, and key set (transitivity: base case $|p| = 1$ by Spec-Prefix on the sole element; inductive step by Spec-Prefix when heads differ, Spec-Tail when heads are level-equal with the inductive hypothesis on the remainder; Spec-Keys from set-containment). Across levels, irreflexivity ensures at least one strict level inequality along any $p_1 \prec p_2 \prec p_3$ chain, so Spec-Level or within-level rules give $p_1 \prec p_3$. Incomparability under $\prec$ arises when (i) patterns differ only in variable names, or (ii) sequential and map patterns share an arity. In both cases $\prec_{\text{disp}}$ falls back to idx , a monotonically increasing counter (re-evaluation increments it), so no two live clauses share an index and $\prec_{\text{disp}}$ is total. Case (ii) is a modularity hazard: sequential and map clauses for the same generic function from different libraries are ordered solely by ASDF load sequence. $\square$

Predicate dispatch requires an $O(N)$ linear scan per call, partitioned by arity group. FOL sorts methods once at load time in $O(ND \log N)$, sacrificing $O(1)$ cached dispatch—which subsumption systems buy with an $O(N^2)$ load-time cost—for syntactic tractability.

4.1.2 *Dispatch Examples.* Listing 1 shows a two-clause defn transpiled to a defun with a static cond cascade.

Listing 1: Transpilation: specificity-ordered multi-clause defn

```
;; FOL source: type specializer (Level 2) before wildcard (Level 0)
(defn classify
  ([x <integer>] :integer)
  ([x] :other))

;; Generated Common Lisp: ordering preserved as cond cascade
;; (class references bound once via load-time-value)
(defun classify (x)
  (cond
    ((typep x (load-time-value (find-class '<integer>))) :integer)
    (t :other)))
```

Comparison to defmulti. Clojure’s defmulti [16] requires manual resolution via prefer-method; FOL’s total order eliminates ambiguity for patterns with distinct types or structures.

4.1.3 *Usage Patterns.* Four patterns illustrate these features: structural diff (Listing 2); validated update guard (Listing 3); predicate dispatch on persistent state (Listing 4); and content routing (Listing 5).

Listing 2: Automatic Structural Diff (FOL)

```
(defmethod assoc :around [(obj <diffable>) key val]
  (if (= key :_changes)
    (call-next-method) ; key=:_changes: breaks cycle
    calls assoc ; directly, skipping the branch

    that emits ; (assoc ... :_changes ...) and re-
    invokes :around
    (bind [old (get obj key)
           new-obj (call-next-method)]
      (if (= old (get new-obj key))
        new-obj
        (assoc new-obj :_changes (inc (get obj :_changes 0)))))))
```

Listing 3: Validated Update Guard (FOL)

```
(defmethod assoc :around [(obj <guarded>) key val]
  (if (valid-update? obj key val)
    (call-next-method)
    obj)) ; discard invalid update; original snapshot unchanged
```

Listing 4: Predicate Dispatch on Persistent State (FOL)

```
;; Level-3 predicate (critical?) tests the reading value at dispatch
time;
;; the body reads persistent slot :alert-history from the persistent
sensor
;; object without copying. Both contributions are required:
predicate
;; dispatch gates the clause; persistent immutability makes prior
history
;; a first-class value.
(defmethod record-reading :around
  [(sensor <sensor>) (reading (critical?))]
  (bind [history (get sensor :alert-history [])
        result (call-next-method)]
    (assoc result :alert-history (conj history reading)))))
```

Listing 5: Value-Based Alerting (FOL)

```
(defmethod notify [{:keys [(temp (>= 100))]]]
  (print "ALERT: Critical Temperature!"))
```

Predicate arguments are closed over at compile time: `(>= 100)` produces `(lambda (temp) (>= temp 100))`.

FOL’s validated guard avoids the explicit wrapper required in Clojure (Listing 6) or the $O(N)$ copy required in standard Common Lisp (Listing 7); `:around` integrates with the existing class specializer without restructuring the dispatch hierarchy. (Clojure’s `:pre`/`:post` conditions cannot discriminate by argument type or class—all callers receive the same check—unlike FOL’s per-class `:around`.)

Listing 6: Validated Update Guard in Clojure — callers must use wrapper

```
;; Guard is in a wrapper; every call site must be changed from
;; (update! obj key val) to (guarded-update! obj key val):
(defn guarded-update! [obj key val]
  (if (valid-update? obj key val)
    (update! obj key val)
    obj))
;; Call sites: (update! ...) -> (guarded-update! ...) ; manual at
each site
```

Listing 7: Validated Update Guard in Common Lisp — requires explicit copy

```
(defmethod update-obj :around ((obj guarded) key val)
  ;; No persistent objects: must copy before calling primary to
  preserve
  ;; original for potential discard. Cost:  $O(N)$  in number of slots.
  (let ((saved (make-instance (class-of obj)
                              :slot-a (slot-value obj 'slot-a)
                              :slot-b (slot-value obj 'slot-b))))
    (if (valid-update-p obj key val)
        (call-next-method)
        saved)))
```

FOL centralizes the guard and discards updates in $O(1)$ via persistence.

4.1.4 Design Limitations. (1) The linear scan evaluates all non-matching predicates before a match; Level-3 predicates should be pure. (2) Load-order sensitivity arises for incomparable patterns (mixed Level-3 or structural types); collapse these into single clauses or use explicit `:around` methods. (3) Open dispatch requires runtime scanning and lacks the reachability analysis possible for closed dispatch. To mitigate load-order sensitivity, the FOL MOP is being extended with a load-time check that warns when two Level-3 predicates are structurally incomparable—i.e., their syntactic forms share no Spec-Prefix, Spec-Tail, or Spec-Keys ordering—at method-definition time.

4.2 Schema Evolution

Chain replay (`:after on update-instance-for-redefined-class`) lazily composes version-chain alias maps and recovers renamed values in a single pass. Migration is confluent if no slot name appears as both source and target: this rules out cycles (which would make chain replay non-terminating) and diamond conflicts (two sources aliased to the same target, making the composed map ambiguous). Macro-expansion enforces these checks when the `defclass`

form is processed at load time. These checks apply to source-level `defclass` forms; programmatic construction bypasses them.

Listing 8: Lazy Schema Migration with Aliasing

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val; :reading keyword still routes to
val
(defclass <sensor> [] [[id] [val :alias reading]])
```

At first post-redefinition *mutation*, chain replay writes `:reading`’s value into `:val`; reads of `:reading` before that mutation resolve via the alias map without modifying the instance.

4.2.1 Persistence and Versioning Performance. Versioning overhead is $O(H \cdot A)$ (H = redefinitions, A = aliases per hop); measured first-touch cost is 3.6–8.2 μ s per instance, not growing linearly with hop depth, (Tables 9–10), paid once. Post-migration *reads* proceed at the standard 1.2–2.0 $\times$ rate; post-migration *updates* remain at the general persistent-object overhead (Table 6), consistent with the 0.77 μ s steady-state update shown in Table 9.

5 Evaluation**5.1 Common Lisp and Clojure Comparison**

FOL fills a gap between CLOS and Clojure (Table 5): FOL provides MOP-integrated persistence and full CLOS method combinations unavailable via `defrecord`.

Table 5: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Static type safety	×	×	×
Persistent objects	✓ ^d	○ ^c	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	single ^a	×
Schema evolution	✓ ^e	×	Manual
Transducers	✓	✓	×
Macros	✓	✓	✓
Raw scalar performance	Low ^f	High	High

^a `defmulti` dispatches via a user-supplied function; `prefer-method/derive` allow manual overrides but there is no automatic multi-parameter specificity ordering.

^b Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.

^c Clojure has persistent *data structures*; `defrecord` is map-backed, not MOP-integrated.

^d Slot storage backed by immutable HAMTs; (`setf slot-value`) blocked; updates via `assoc`.

^e Renamed slots recovered automatically at first post-redefinition access; multi-hop chains composed in a single pass.

^f Scalar arithmetic and non-slot operations inherit full CL performance. “Low” refers specifically to persistent object slot access: reads add 1.2–2.0 $\times$ overhead and writes add 30–248 $\times$ overhead relative to mutable CLOS.

5.2 Micro-Benchmarks

We measured persistent objects vs. mutable CLOS across slot counts. **Construction** (≈ 183 ns) is 5 $\times$ native; `%make-persistent` bypasses `make-instance` keyword-argument dispatch while still routing

through `initialize-instance`, which stamps `%schema-version` and enforces the persistence invariant. **Reads** add 1.2–2.0× overhead; **Updates** are 30–248× slower. The modeled update cost (Table 7) shows $T = 8$ is optimal at $N = 8$ and $N \geq 32$; at $N = 24$ it ties with $T = 24$ and $T = 32$, and yields to higher thresholds at $N = 12$ –16. Because $\approx 80\%$ of classes have ≤ 8 slots [21], $T = 8$ maximises coverage of the fully-native path; wider classes where larger thresholds help are rare in practice.

Table 6: Actual slot update overhead ($\mu\text{s/op}$) vs. native CLOS

Slots	Persistent	Mutable	Ratio	Notes
2	0.58 μs	14.1 ns	41×	2 native slots
8	1.03 μs	13.4 ns	77×	8 native slots
32	1.08 μs	26.8 ns	40×	8N + 24V trie
48	2.23 μs	24.0 ns	93×	8N + 40V trie
128	4.00 μs	16.1 ns	248×	8N + 120V trie

^a Ratio drops vs. 32 slots because the mutable CLOS baseline is 46% slower at 48 slots (discontiguous memory layout), not because persistent overhead decreases.

Table 7: Modeled update cost ($\mu\text{s/op}$) at candidate thresholds T

N	T=8	T=12	T=16	T=24	T=32	T=8 strategy
8	0.42	0.42	0.43	0.42	0.42	all-native
12	0.75	0.62	0.62	0.61	0.60	8N + 4V
16	0.94	0.91	0.83	0.78	0.77	8N + 8V
24	1.15	1.23	1.28	1.15	1.15	8N + 16V
32	1.39	1.47	1.51	1.58	1.52	8N + 24V
48	1.95	1.99	2.04	2.12	2.22	8N + 40V

Bold = minimum in row. “ $kN + jV$ ” = copy k native slots then batch-build j -element overflow vector. All-native = copy N slots directly. Simulated copy costs only; actual update-slots is 1.6–3× higher due to MOP iteration overhead (see Table 6).

5.2.1 Predicate Dispatch Micro-benchmark. We measured the per-invocation cost of a 20-method generic function, each clause specialized on a distinct Level-3 predicate, comparing FOL open and closed dispatch against standard CLOS type dispatch and a manual `cl:cond cascade` (Table 8, SBCL 2.6.0 x86-64).

Table 8: Predicate Dispatch Performance (N=20 clauses).

Dispatch Method	Time/op	Ratio (vs. CLOS)
Standard CLOS (Type)	37.4 ns	1.00×
Manual <code>cl:cond Cascade</code>	62.4 ns	1.66×
FOL defn (Closed Dispatch)	60.0 ns	1.60×
FOL defgeneric (Open Dispatch)	64.4 ns	1.72×

FOL’s predicate dispatch adds $\approx 1.6\times$ overhead, matching the manual `cl:cond` baseline; the 20-clause all-Level-3 setup approximates a worst case. Functions with only Level-2 specializers use CLOS type dispatch directly and incur no predicate scan; this path

is not separately benchmarked but is structurally identical to standard CLOS dispatch. Since each predicate evaluation is a function call, overhead scales linearly with N ; at $N = 50$ it would approach $\approx 4\times$ CLOS.

5.2.2 Lazy Evolution Micro-Benchmark. We measured single-hop and multi-hop migration against a CL baseline with a manual `update-instance-for-redefined-class` method. FOL single-hop first-touch cost is $\approx 2\times$ CL’s; multi-hop cost (3.6–8.2 μs) does not grow linearly with hop depth, as chain-replay composes alias maps in a single pass rather than scanning each version incrementally (Tables 9–10).

Table 9: Single-hop lazy migration: FOL vs. CL ($\mu\text{s/op}$, 10K instances)

Phase	FOL	CL	Notes
Migration (first touch)	6.40	3.30	one-time per dormant instance
of which:	5.63	3.28	SBCL migration + alias recovery
overhead vs. steady-state			
Steady-state update	0.77	0.02	post-migration, repeated use

Table 10: Multi-hop lazy migration first-touch cost ($\mu\text{s/op}$, 5K instances per cohort)

Hop depth	FOL	CL	Notes
1-hop ($v_2 \rightarrow v_3$)	8.19 ^b	4.10	FOL allocates new object; CL mutates in-place
2-hop ($v_1 \rightarrow v_3$)	3.57	5.62	CL plist scan grows $O(A \cdot H)$; FOL folds alias maps left-to-right in a single pass—each slot name resolved once—giving $O(A)$ regardless of hop depth
3-hop ($v_0 \rightarrow v_3$)	4.48	5.13	SBCL migration machinery dominates both; FOL retains its $O(A)$ alias-map advantage over CL’s $O(A \cdot H)$ scan, yielding a small but consistent lead
Steady-state	0.69	0.01	post-migration; uniform across all cohorts

^b Higher than Table 9’s 6.40 μs : instances created at v_2 carry two prior class redefinitions in SBCL’s version history, incurring more migration overhead than instances created at v_1 or v_0 regardless of FOL hop depth. This explains the apparent anomaly that 1-hop costs more than 2-hop.

5.3 Naive-Translation Workloads

Persistent *slot* updates are 33–447× slower than mutation (Table 6). Naive Quicksort and BFS (Listings 9–10) illustrate the path-copying penalty (Table 11).

Quicksort overhead stems from 3.4M path copies; transients cut this from 96× to 40× (-97% allocation). BFS improves similarly (39× $\rightarrow$ 6.2×).

Listing 9: Naive Persistent Partition

```
(defn partition [v low high]
  (bind [pivot (get v high)]
    (loop [j low i (dec low) curr-v v]
      (if (< j high)
        (if (<= (get curr-v j) pivot)
          (bind [next-i (inc i)]
            v1 (assoc curr-v next-i (get curr-v j))
            v2 (assoc v1 j (get curr-v next-i)))
          (recur (inc j) next-i v2))
        (recur (inc j) i curr-v))
      (bind [next-i (inc i)]
        v1 (assoc curr-v next-i (get curr-v high))
        v2 (assoc v1 high (get curr-v next-i)))
      [v2 next-i])))
```

Listing 10: Naive Persistent BFS Step

```
(defn bfs-step [q dists graph]
  (if (empty? q)
    dists
    (bind [u (first q)]
      edges (get graph u)
      d (get dists u)
      (loop [es edges nq (rest q) nd dists]
        (if (empty? es)
          (bfs-step nq nd graph)
          (bind [v (first es)]
            (if (= (get nd v) -1)
              (recur (rest es)
                (conj nq v)
                (assoc nd v (inc d)))
              (recur (rest es) nq nd)))))))
```

Table 11: Naive-Translation Workload Results

Workload	Implementation	Time	Alloc	vs. CL
QuickSort $N = 100,000$	CL (simple-vector)	12 ms	0.8 MB	1.0×
	FOL Persistent (assoc)	1199 ms	2598 MB	~96×
	FOL Transient (assoc!)	494 ms	91 MB	~40×
BFS $N = 50,000$	CL (hash-table)	4 ms	2.3 MB	1.0×
	FOL Persistent (assoc)	148 ms	119 MB	39×
	FOL Transient (assoc!)	24 ms	13 MB	6.2×

Idiomatic Reformulations. Algorithmic restructuring eliminates the translation penalty (Table 12). **Idiomatic merge sort** uses subvec and conj to minimize copying. **Lazy BFS** initializes nodes on discovery, bypassing N initial assoc calls.

Merge sort’s 1.7M trie-node touches (near $O(N \log^2 N)$) make transients counterproductive: the per-conj path is already shallow (depth ≤ 5), so the fixed cost of acquiring and committing a transient handle—thread-local ownership check plus dirty-node finalization—is not recovered at $N = 100,000$. Lazy BFS eliminates initialization waste; transients help further ($29\times \rightarrow 14\times$).

5.4 Abstract Syntax Tree Rewriting Engine

To evaluate both contributions together, we implemented an AST optimization engine: 25 node classes, a single multi-clause defmethod

Table 12: Idiomatic Algorithm Results: persistent and transient FOL vs. native CL

Workload	Implementation	Time	Alloc	vs. CL
Merge Sort $N = 100,000$	CL (simple-vector)	15 ms	1.6 MB	1.0×
	FOL persistent (conj)	457 ms	623 MB	30×
	FOL transient (HAMT array)	491 ms	81 MB	32× ^a
BFS lazy $N = 50,000$	CL (hash-table)	4 ms	2.3 MB	1.0×
	FOL persistent (lazy dict)	103 ms	61 MB	29×
	FOL transient (hamt-assoc!)	51 ms	16 MB	14×

^a Transient is slower than persistent here: persistent conj touches only a root-to-leaf path of depth ≤ 5 per append, so transient setup cost is not recovered at $N = 100,000$.

encoding 54 algebraic rewrite rules via predicate dispatch (including nested structural patterns such as the identity-element rule below), and a 25-clause walk driving recursive descent.

```
(defmethod ast-optimize
  [({:keys [(left ({:keys [(val (eq1 0))]} <op-lit>))
            (right r)]} <op-add>)]
  r)
```

The built-in eq1 specialization (`(val (eq1 0))`) expands to `(lambda (x) (eq1 x 0))`, matching a slot against a specific value; it appears in the grammar’s predicate position. The CL baseline uses defstruct with typecase on a 9,999-node balanced tree.

Table 13 summarizes results. FOL is 43× slower per optimizer pass (14× more memory), driven by dispatch routing, slower field reads, and object construction. The build ratio (78×) exceeds the optimizer ratio because build requires one `make-<class>` per node with no amortization; transient builders (Section 3) are expected to reduce this.

Table 13: AST Optimizer: FOL vs. CL ($N = 100$ passes, 9,999-node balanced tree).

Phase	Implementation	Time	Alloc	vs. CL
Build	CL	0.398 ms	0.16 MB	1.0×
	FOL	31.1 ms	12.7 MB	78×
Optimizer (per pass)	CL	0.240 ms/pass	0.125 MB/pass	1.0×
	FOL	10.36 ms/pass	1.749 MB/pass	43×

5.5 Logic Simulation (LSim)

LSim is an event-driven digital logic simulator that exercises persistent data structures at scale. To isolate the event-queue cost, we hold gate-connectivity constant: the **CL** baseline uses a mutable destructive leftist heap [8] ($O(1)$ allocation per merge); **FOL** uses a persistent leftist heap [24] ($O(\log n)$ copies). Table 14 covers six configurations spanning five orders of magnitude; the largest circuits used a 56 GB heap (`-dynamic-space-size 57344`).

At scale, CL mutable heap fragmentation degrades locality; path-copying avoids this, yielding FOL’s 3.8× advantage.

A 2×2 ablation (Table 15) breaks down these costs.

Both persistent-map and persistent-heap overheads peak at mid-scale (5–6×) and fall at the extremes; at 8x32x32-9000, FOL matches Hybrid-A (983 s) despite Hybrid-B reaching 1646 s. At the unablated

Table 14: LSim PQ Scaling Results (mean CPU time).

Config	Gate Evals	CL	FOL	Ratio (FOL/CL)
8bit-100	876	78 ms	109 ms	1.4×
32bit-300	10 224	94 ms	141 ms	1.5×
8x32-9000	281 248	359 ms	1.87 s	5.2×
32x32-3000	3 850 304	7.57 s	34.15 s	4.5×
8x32x32-9000	89 708 032	906 s	983 s	1.1×
32x32x32-30000	1,183,510,528	52,050 s	13,531 s	0.26× ^a

^a Ratio < 1 means FOL is faster than CL; 0.26× = FOL is 3.8× faster. The largest config was not ablated (Table 15); the fragmentation explanation is mechanistic, not derived from the 2×2 breakdown.

Table 15: 2×2 Ablation: Mutable vs. Persistent Heap × State Maps (mean wall time; ratios relative to CL baseline). Hybrid-A = persistent maps + mutable heap; Hybrid-B = mutable maps + persistent heap; FOL = both persistent.

Circuit	CL	Hybrid-A	Hybrid-B	FOL
32bit-300	94 ms	141 ms (1.5×	125 ms (1.3×	141 ms (1.5×
8x32-9000	359 ms	1.87 s (5.2×	2.25 s (6.3×	1.87 s (5.2×
32x32-3000	7.57 s	34.1 s (4.5×	42.7 s (5.6×	34.1 s (4.5×
8x32x32-9000	906 s	983 s (1.1×	1646 s (1.8×	983 s (1.1×

32x32x32-30000 configuration, FOL ran 3.8× faster; the ablation does not cover this config, so the gain is attributed to heap fragmentation on mechanistic grounds rather than direct decomposition. A CL baseline with a slab allocator might narrow this gap but remains outside our scope.

5.6 CLOS Adoption Cost: Expression Interpreter

We port an idiomatic CLOS expression interpreter (seven node types, three generic functions: `eval-expr`, `pretty`, `free-vars`) to FOL with minimal structural changes. The CL baseline uses mutable environments; FOL uses (`assoc env bvar v`); method bodies are otherwise identical.

A corpus of 50 depth-5 expression trees is evaluated across 1,000 iterations (150,000 total call sequences).

Results. The 7× eval ratio (vs. 43× for the AST optimizer) reflects: a make-instance baseline ($\approx 2\times$ construction vs. $5\times$ for `defstruct`), 7-clause scans rather than 25, and slot-read-heavy dispatch. The port requires no architectural changes; mechanical differences are `defclass` syntax, slot reads, constructors, environment management, and collection APIs.

6 Threats to Validity

Single implementation: All measurements use SBCL 2.6.0; results may differ on other CL implementations. **Experimenter bias:** FOL and all baselines share the same author; more aggressive CL tuning might narrow results. **Benchmark coverage:** Smaller LSim configs were run repeatedly, but the largest circuit was run only once per implementation due to its scale (≈ 14 hours). **Benchmark selection bias:** Macro-benchmarks may over-represent FOL’s advantages; the interpreter and naive-translation benchmarks offer a more conservative view. **Qualitative comparison:** Section 4

compares code-level conciseness but does not measure programmer productivity. **Interoperability hazard:** CL authors extending FOL classes may encounter runtime mutation errors; these are not signaled at method-definition time. **Non-portability:** The persistent-class metaclass relies on SBCL-specific MOP hooks; portability to other runtimes (e.g., CCL, ECL, ABCL) is untested. **Dispatch scaling:** The predicate dispatch micro-benchmark uses a single $N = 20$ configuration; overhead is projected to scale linearly with N , but this scaling is not empirically verified across multiple clause counts.

7 Related Work

Persistent data structures. FOL’s trie follows Bagwell’s HAMT [3] and Clojure [16], using path-copying persistence [10]; Okasaki’s heaps [24] underpin our LSim heap. FOL extends libraries like Sycamore [9] and FSet [5] to persistent *objects* with predicate dispatch.

Schema evolution. Gemstone [27] and ENCORE [29] pioneered lazy migration, but require explicit coercion code for renames [2]. FOL provides declarative migration via `:alias`; multi-hop chains compose in a single pass via the CLOS MOP.

OOP/FP bridges and pattern dispatch in other languages. Scala’s case classes [23] and `match` [11] provide immutable ADTs with structural dispatch, but updates require explicit copy and the class system is not MOP-integrated. Racket [13] separates `match` from its class system [14]; Julia [4] provides multi-parameter type dispatch (FOL’s Level 2) but no predicate specializers and mutable objects.

Expression Problem solutions. Object algebras [25] and tagless-final style [6] solve both dimensions in typed languages with static guarantees. FOL’s approach is a multimethod-based solution [32] that trades static safety for CLOS expressibility and open-world extension.

Predicate dispatch. Predicate dispatch [7, 12] requires theorem-prover subsumption for method ordering; FOL’s syntactic level ordering avoids this $O(N^2)$ cost at the price of semantic exactness. JPred [22] uses domain containment checks, fragile in CLOS’s open-world model. Our proposed composite level (Section 4) recovers ordering for type-annotated predicates without a theorem prover. GHC’s overlapping instances [15] order heads via pragmas but lack a predicate stratum or schema evolution. Filtered dispatch [26] introduced guard predicates as first-class criteria, motivating FOL’s Level-3. Dylan’s singletons [1] and Slate’s dispatch [28] are restricted forms. Clojure’s `defmulti` [16] dispatches via a user-supplied function with `prefer-method` for ambiguity; *protocols* [17] dispatch on the first argument’s class only. FOL’s `defmethod` occupies the same niche as `defmulti` but adds predicate specializers and CLOS method combinations, trading an $O(N)$ scan for expressivity. Self’s [30] “slots as map entries” model influences FOL’s object representation.

Table 16: Expression Interpreter Benchmark (SBCL 2.6.0, AMD Ryzen 9 5900X).

Phase	CL	FOL	Time ratio	Mem ratio
Build (50 trees, depth 5)	9.6 ms / 3.8 MB	19.9 ms / 8.9 MB	2.1×	2.3×
Eval (1000 iter × 50 exprs)	0.27 s / 224.5 MB	1.88 s / 1300 MB	7.0×	5.8×

8 Conclusion

We presented two contributions to the design of Lisp object systems, supporting open extension and addressing the new-operations dimension of the Expression Problem within a dynamically typed setting. First, a **persistent metaclass system** built on the CLOS MOP: hybrid native/trie storage keeps $\approx 80\%$ of classes on the 1.2–2.0× read-overhead native path; lazy alias-based migration delivers transparent instance evolution; and %make-persistent reduces object creation to 5× over mutable make-instance. Second, a **specificity-ordered predicate dispatch** whose load-time-decidable $\prec_{\text{disp}}$ orders all structurally or type-differentiated patterns, confining load-order sensitivity to incomparable Level-3 predicates.

Slot updates cost 30–248× in isolation; at 1.18 billion gate evaluations, FOL is 3.8× faster as the CL heap degrades. The AST optimizer pass is 43× slower; speculative rollback is $O(1)$ by construction—discarding an assoc result returns the original snapshot without copying; the speculative assoc itself costs $O(\log n)$ —but is not separately benchmarked.

Open problems include: native compilation to replace the $O(N)$ predicate scan; static clause analysis to detect unreachable patterns at load time; and coordinated value transitions across agent networks with snapshot consistency.

FOL’s implementation is available at <https://github.com/frankadrian314159/fol>.

References

- [1] Apple Computer, Inc. 1992. *Dylan - An Object-Oriented Dynamic Programming Language*. Apple Computer, Inc., Cupertino, CA, USA.
- [2] Malcolm Atkinson and Ron Morrison. 1995. Orthogonally Persistent Object Systems. *The VLDB Journal* 4, 3 (1995), 319–401.
- [3] Phil Bagwell. 2001. *Ideal Hash Trees*. Technical Report 1. EPFL, Lausanne, Switzerland.
- [4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev* 59, 1 (2017), 65–98.
- [5] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. <https://common-lisp.net/project/fset/>
- [6] Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. 2009. Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages. *Journal of Functional Programming* 19, 5 (2009), 509–543.
- [7] Craig Chambers. 1993. The Cecil Language: Design and Performance. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications*. ACM, New York, NY, USA, 243–257.
- [8] Clark Allen Crane. 1972. *Linear Lists and Priority Queues as Balanced Binary Trees*. Technical Report STAN-CS-72-259. Stanford University.
- [9] Neil T. Dantam. 2013. Sycamore: Purely Functional Data Structures for Common Lisp. <https://github.com/ndantam/sycamore>
- [10] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.
- [11] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007—Object-Oriented Programming*. Springer, Berlin, Heidelberg, 273–298.
- [12] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP’98—Object-Oriented Programming*. Springer, Berlin, Heidelberg, 186–211.
- [13] Matthew Flatt et al. 2015. The Racket Manifesto. In *20 Years of the Past and 20 Years of the Future (Snapshots)*, Dagstuhl Seminar 15451. <https://doi.org/10.4230/DagRep.5.11.112>
- [14] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.
- [15] GHC Team. 2004. *GHC User’s Guide: Overlapping Instances*. Glasgow Haskell Compiler. https://downloads.haskell.org/ghc/latest/docs/users_guide/.
- [16] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. ACM, New York, NY, USA, 1–10.
- [17] Rich Hickey. 2010. Clojure Protocols. Clojure Reference Documentation. <https://clojure.org/reference/protocols>
- [18] Rich Hickey. 2010. Clojure’s Solutions to the Expression Problem. Conference talk at Strange Loop. Available at [thestrangeloop.com](https://www.thestrangeloop.com/2010/clojures-solutions-to-the-expression-problem.html).
- [19] Rich Hickey. 2014. Transducers. Clojure Blog. <https://clojure.org/reference/transducers>
- [20] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.
- [21] Michele Lanza and Radu Marinescu. 2006. *Object-Oriented Metrics in Practice*. Springer.
- [22] Todd Millstein. 2004. Practical Predicate Dispatch. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 345–364.
- [23] Martin Odersky, Lex Spoon, and Bill Venners. 2016. *Programming in Scala, 3rd Edition*. Artima Press.
- [24] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.
- [25] Bruno C. d. S. Oliveira and William R. Cook. 2012. Extensibility for the Masses: Practical Extensibility with Object Algebras. In *Proceedings of the 26th European Conference on Object-Oriented Programming (ECOOP)*. Springer, 2–27.
- [26] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. ACM, New York, NY, USA, 20–26.
- [27] D. J. Penney and J. Stein. 1987. Class Modification in the GemStone Object-Oriented DBMS. In *Proceedings of the 2nd ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 111–117.
- [28] Lee Salzman and Jonathan Aldrich. 2005. Prototypes with Multiple Dispatch: An Expressive and Dynamic Object Model. In *ECOOP 2005—Object-Oriented Programming*. Springer, 312–336.
- [29] Andrea H. Skarra and Stanley B. Zdonik. 1986. The Management of Changing Types in an Object-Oriented Database. In *Proceedings of the 1st ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 483–495.
- [30] David Ungar and Randall B. Smith. 1987. Self: The Power of Simplicity. *ACM SIGPLAN Notices* 22, 12 (1987), 227–242.
- [31] Philip Wadler. 1990. Linear Types Can Change the World!. In *Programming Concepts and Methods*. North-Holland, Amsterdam, Netherlands, 347–359.
- [32] Matthias Zenger and Martin Odersky. 2005. Independently Extensible Solutions to the Expression Problem. In *Proceedings of the 12th International Workshop on Foundations of Object-Oriented Languages (FOOL)*.